
DOCKER JOURNEY:
Beginner to Advanced DevOps
1. KEY CONCEPTS
Docker
Containers
Dockerfile
>_
> run
> pull
> ps
2. BEGINNER COMMANDS
3. INTERMEDIATE SKILLS
docker-compose.yml
Swarm
Volume
4. ADVANCED TECHNIQUES
Secrets
Inspect
Prune
5. BEST PRACTICES
CI/CD Automation

1. Architecture Difference (Visual understanding)

🔹 **Virtual Machine (Virtualization)**

Hardware → Hypervisor → VM (Guest OS + App + Libraries)

- Each VM has its **own full OS**
- Uses a **hypervisor** (VMware, VirtualBox)

🔹 **Container (Docker)**

Hardware → Host OS → Docker Engine → Containers (App + Libraries only)

- **No separate OS per container**
- All containers **share host OS kernel**

⚖️ 2. Key Differences Table

Feature	Virtualization (VM)	Containers (Docker)
Level	Hardware-level	OS-level
OS	Each VM has its own OS	Share host OS
Size	Heavy (GBs)	Lightweight (MBs)
Startup	Slow (minutes)	Fast (seconds)
Performance	More overhead	Near-native performance
Isolation	Strong	Moderate
Portability	Lower	Very high
Example	VMware, VirtualBox	Docker, Kubernetes

✅ VMs emulate **entire machine**

✅ Containers package **only application + dependencies** [learn.microsoft.com], [geeksforgeeks.org]

📦 3. Docker Image vs Container (Important!)

🔹 Docker Image

- **Read-only blueprint/template**
- Contains:
  - App code
  - Libraries
  - Dependencies
- Cannot change once built

👉 Think: **Class / Blueprint**

🔹 Docker Container

- **Running instance of image**
- When you execute:

Shell

docker run image_name

Show more lines

- It becomes a live process

👉 Think: **Object / Running program**

📌 A container = image + runtime layer [geeksforgeeks.org]

🧠 4. Simple Analogy

- **Virtualization (VM)** → 🏠 Independent Houses
  Each house has its own kitchen, plumbing (OS)
- **Containers** → 🏢 Apartments
  Shared building (OS), separate rooms (apps)

🎯 5. When to Use What

✅ **Use Virtual Machines when:**

- Need **different OS** (Windows + Linux together)
- Need **strong security isolation**
- Running legacy systems

✅ **Use Containers when:**

- Building **microservices**
- Need **fast scaling**
- Want **CI/CD pipelines**

💡 Quick Summary

- VM = **Heavy, full OS, strong isolation**
- Container = **Lightweight, fast, share OS**
- Docker Image = **Template**
- Container = **Running instance**

If you want, I can convert this into **short notes (1–2 page exam revision)** or **diagram you can paste into Word**.

Sources

Provide your feedback on BizChat

You said:

yes

Copilot said:

Copilot

Here's a **clean revision note + diagram** (perfect for your Word notes 👇):

📝 **Virtualization vs Containers (with Diagram + Image concept)**

📊 **Architecture Diagram**

🔴 **Virtualization (VM)**

```
+----------------------+
|  App + Libraries    |
+----------------------+
|    Guest OS         |
+----------------------+
|    Hypervisor       |
+----------------------+
|  Physical Hardware  |
+----------------------+
```

📌 Key idea:
👉 Each VM has **its own OS**

🟢 **Containers (Docker)**

```
+----------------------+
| App + Libraries     |
+----------------------+
| App + Libraries     |
+----------------------+
| App + Libraries     |
+----------------------+
| Docker Engine       |
+----------------------+
```

| Host OS (Shared) |

+--------------------+

| Physical Hardware |

+--------------------+

📌 Key idea:

👉 All containers **share the same OS**

⚖️ **Quick Comparison (Exam Table)**

Feature	Virtualization	Containers
Type	Hardware-level	OS-level
OS	Separate OS per VM	Shared OS
Size	Heavy (GB)	Light (MB)
Startup	Slow	Fast
Isolation	Strong	Medium
Performance	Lower (overhead)	High
Example	VMware	Docker

📦 **Docker Image Concept**

🔹 **Docker Image**

- Blueprint (read-only)
- Contains app + dependencies
- Created using Dockerfile

👉 Example:

Node.js + App Code + Libraries

🔹 **Docker Container**

- Running instance of image
- Created using:

docker run image_name

🧠 **Easy Memory Trick**

Concept	Analogy
VM	🏠 Separate Houses
Container	🏢 Apartments
Image	📐 Blueprint
Container (runtime)	🏃 Running house

🎯 **1-Line Summary (Exam Gold ⭐)**

👉 **VM = Full OS + heavy + strong isolation**
👉 **Container = No OS + lightweight + fast**
👉 **Image = Template, Container = Running instance**

Docker Commands

From Beginner to Advanced for DevOps Engineers

Introduction

Docker is a platform for developing, shipping, and running applications insidecontainers.Itprovidesa **lightweight, portable, and consistent environment for software development and deployment**. This guide covers essential **Docker commands from beginner to advanced levels**, helping DevOps engineers effectively manage their **containerized applications**.

Key Concepts

Before diving into the commands, let's review some fundamental Docker concepts:

- **Image**: A lightweight, standalone, and executable package that includes everything needed to run a piece of software.
- **Container**: A runtime instance of a Docker image.
- **Dockerfile**: A script containing a series of instructions on how to build a Docker image.
- **Docker Compose**: A tool for defining and running multi-container Docker applications.

Beginner Commands

1. Installation

Check Docker Version

```
docker --version
```

Displays the installed version of Docker.

2. Docker Images

List Docker Images

```
docker images
```

Lists all Docker images on the local machine.

Pull Docker Image

```
docker pull <image_name>
```

Pulls a Docker image from a registry (e.g., Docker Hub).

Remove Docker Image

```
docker rmi <image_name>
```

Removes a Docker image from the local machine.

3. Docker Containers

List Running Containers

```
docker ps
```

Lists all running Docker containers.

List All Containers

```
docker ps -a
```

Lists all Docker containers, including stopped ones.

Run a Container

```
docker run -d --name <container_name> <image_name>
```

Runs a container from a Docker image in detached mode.

Stop a Container

```
docker stop <container_name>
```

Stops a running Docker container.

Start a Container

```
docker start <container_name>
```

Starts a stopped Docker container.

Remove a Container

```
docker rm <container_name>
```

Removes a Docker container.

View Container Logs

```
docker context ls
```

Lists all Docker contexts.

Create Docker Context

```
docker context create <context_name>
```

Creates a new Docker context.

Use Docker Context

```
docker context use <context_name>
```

Switches to a specific Docker context.

8. Docker BuildKit

Enable BuildKit

```
export DOCKER_BUILDKIT=1
docker build -t <image_name> .
```

Enables Docker BuildKit for improved build performance.

Best Practices

Use Version Control

- Store your Dockerfiles, Compose files, and other configurations in a version control system (e.g., Git) to track changes and

```
docker import <file_name>.tar
```

Imports a container from a tar file.

5. Docker Save and Load

Save Docker Image

```
docker save -o <file_name>.tar <image_name>
```

Saves a Docker image to a tar file.

Load Docker Image

```
docker load -i <file_name>.tar
```

Loads a Docker image from a tar file.

6. Docker Prune

Remove Unused Data

```
docker system prune
```

Removes unused Docker data (images, containers, networks, volumes).

7. Docker Context

List Docker Contexts

Inspect Docker Image

```
docker inspect <image_name>
```

Displays detailed information about a Docker image.

2. Docker Stats

View Container Statistics

```
docker stats <container_name>
```

Displays real-time statistics for Docker containers.

3. Docker Events

View Docker Events

```
docker events
```

Displays real-time events from the Docker server.

4. Docker Export and Import

Export Container

```
docker export <container_name> -o <file_name>.tar
```

Exports a Docker container to a tar file.

Import Container

```
docker build -t <image_name> .
```

Builds a Docker image from a Dockerfile.

4. Docker Swarm

Initialize Docker Swarm

```
docker swarm init
```

Initializes a Docker Swarm.

List Swarm Nodes

```
docker node ls
```

Lists all nodes in the Docker Swarm.

Deploy Stack in Swarm

```
docker stack deploy -c <compose-file> <stack_name>
```

Deploys a stack in the Docker Swarm.

Remove Stack from Swarm

```
docker stack rm <stack_name>
```

Removes a stack from the Docker Swarm.

Runs the Docker Compose application in detached mode.

Stop Docker Compose

```
docker-compose down
```

Stops and removes the Docker Compose application.

3. Dockerfile

Dockerfile Example

```
# Use an official Python runtime as a parent image
FROM python:3.8-slim

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at
/app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]
```

Defines a Dockerfile for a Python application.

Build Docker Image

5. Docker Secrets

Create Docker Secret

```
echo "my_secret_value" | docker secret create <secret_name>
-
```

Creates a new Docker secret.

List Docker Secrets

```
docker secret ls
```

Lists all Docker secrets.

Remove Docker Secret

```
docker secret rm <secret_name>
```

Removes a Docker secret.

Advanced Commands

1. Docker Inspect

Inspect Docker Container

```
docker inspect <container_name>
```

Displays detailed information about a Docker container.

Install Docker Compose

```
sudo curl -L
"https://github.com/docker/compose/releases/download/<version>
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-
compose
sudo chmod +x /usr/local/bin/docker-compose
```

Installs Docker Compose.

Check Docker Compose Version

```
docker-compose --version
```

Displays the installed version of Docker Compose.

Docker Compose File Example

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "80:80"
  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: example
```

Defines a multi-container application using Docker Compose.

Run Docker Compose

```
docker-compose up -d
```

Intermediate Commands

1. Docker Volumes

List Docker Volumes

```
docker volume ls
```

Lists all Docker volumes.

Create Docker Volume

```
docker volume create <volume_name>
```

Creates a new Docker volume.

Remove Docker Volume

```
docker volume rm <volume_name>
```

Removes a Docker volume.

Mount Volume to Container

```
docker run -d --name <container_name> -v
<volume_name>:/path/in/container <image_name>
```

Mounts a Docker volume to a container.

2. Docker Compose

```
docker logs <container_name>
```

Displays the logs of a Docker container.

4. Docker Networks

List Docker Networks

```
docker network ls
```

Lists all Docker networks.

Create Docker Network

```
docker network create <network_name>
```

Creates a new Docker network.

Connect Container to Network

```
docker network connect <network_name> <container_name>
```

Connects a container to a Docker network.

Disconnect Container from Network

```
docker network disconnect <network_name> <container_name>
```

Disconnects a container from a Docker network.

collaborate with team members.

Keep Images Lightweight

- Minimize the size of your Docker images by using multi-stage builds and removing unnecessary files and dependencies.

Use Tags

- Tag your Docker images with meaningful and version-specific tags to easily identify and manage them.

Secure Images and Containers

- Scan your Docker images for vulnerabilities and use security best practices to protect your containers.

Automate Builds and Deployments

- Integrate Docker with CI/CD pipelines to automate the building, testing, and deployment of your containerized applications.

Monitor and Log

- Continuously monitor your Docker containers and collect logs for troubleshooting and performance analysis.

Use Docker Compose for Multi-Container Applications

- Use Docker Compose to define and manage multi-container applications, making it easier to deploy and scale your services.

Regularly Prune Unused Resources

- Regularly prune unused images, containers, networks, and volumes to free up disk space and maintain a clean Docker environment.